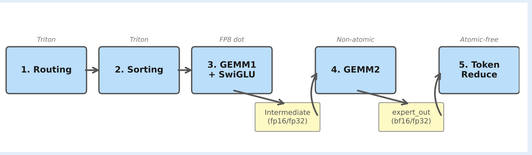
Fused MoE Inference on Blackwell: A Workload-Adaptive Kernel System for DeepSeek-V3

Jiayao Zhang Jiaoliang Yu MLSys 2026 FlashInfer-Bench Challenge — Track A

Motivation

- DeepSeek-V3 MoE: 256 routed experts (32 local per GPU), top-8 routing, H=7168, I=2048; two FP8 block-scaled GEMMs per forward pass
- **Hardware:** NVIDIA B200 (Blackwell, sm_100a) — ∼2,500 TFLOPS FP8, 96 MB L2, 8 TB/s HBM3e
- Naïve per-expert loop: kernel launch overhead dominates small T; atomic scatter hurts large T
- **Goal:** Workload-adaptive fused kernel system covering T=1 to 14,107 with Triton pipeline + CuTe hybrid for largest traces

Five-Stage Pipeline



1. **Routing:** Triton DeepSeek-V3 no-aux routing (sigmoid, group top-4, global top-8, normalization)
2. **Sorting:** Tile-parallel histogram with atomic offsets and padded row construction
3. **GEMM1+SwiGLU:** Fused W_1/W_3 projections in shared K-loop with `tl.dot(fp8,fp8)`
4. **GEMM2:** Non-atomic down-projection → `expert_out` (bf16)
5. **Token Reduce:** 2D tiled accumulation of K=8 expert contributions per output token

Singleton T=1 and large-T CuTe paths reuse the same stages with specialized kernels.

Experimental Setup

- **HW/SW:** B200 (sm_100a), PyTorch 2.12+cu132, Triton 3.7.0
- **Scoring:** CUPTI GPU kernel time only; CPU overhead (routing, sorting, ∼600 μs) excluded from score
- **Tolerance:** atol=1.0, rtol=0.3, 90% ratio — ∼100× wider than our strict validation (atol=0.01)
- **Workloads:** 19 real-trace batches (T=1 to 14,107); same-session A/B testing with ±2% mean noise floor
- **Baseline:** FlashInfer reference MoE kernel (PyTorch-based per-expert dispatch)

Workload-Adaptive Dispatch

Regime	BM	Path
$2 \le T \le 128$	16	Generic Triton
$129 \le T \le 512$	32	Generic Triton
$513 \le T \le 2047$	64	Generic Triton
$T \ge 2048$	128	Generic (exact launch)
T=1	16	Singleton Triton
T=53–59	8	Tight-padding Triton
T=901	64	Isolated Triton
T=11948, 14107	128	Triton/CuTe hybrid

$T \ge 4096$: exact grid from `total_blocks` avoids ∼30% idle blocks. CuTe handles the grouped GEMM core for the largest traces; Triton handles metadata and epilogues.

Key Optimizations

1. FP8 Native Tensor Core Dot — 2× throughput

`tl.dot(fp8, fp8)` on Blackwell `tcgen05.mma`; block-scale dequant *after* the dot. Fused W_1/W_3 gate/up in shared K-loop with inline SwiGLU, eliminating a separate elementwise kernel.

2. Non-Atomic GEMM2 + Token Reduce — +46%

Atomic scatter = 6× BW amplification (12 B FP32 RMW vs. 2 B bf16 store). Two-pass design: GEMM2 writes bf16 `expert_out` (non-atomic); separate token-reduce kernel accumulates K=8 expert contributions per token in FP32. Fused atomic alternative tested and rejected (−4.5% on large T).

3. FP16 Intermediate (×0.125/×8.0) — +4.5%

SwiGLU spans ±40K (exceeds FP16 max 65504). Scale ×0.125 in GEMM1 → FP16 store → ×8.0 in GEMM2. Halves HBM traffic on [MAX_PADDED, 7168] Intermediate. FP32 fallback for T<32.

4. Workload-Adaptive Dispatch

BLOCK_M = 8/16/32/64/128 by token count. Column-major (GROUP_M=32/64) for L2 reuse. T=1: fused singleton kernels. T=53–59: BLOCK_M=8. $T \ge 2048$: exact grid launch. Largest traces: CuTe grouped-GEMM core.

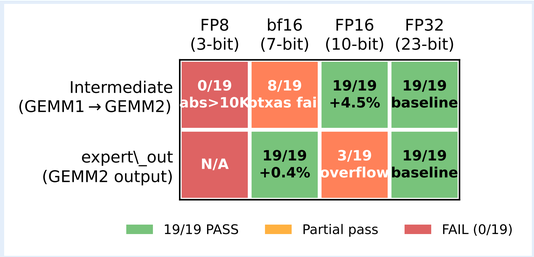
5. Compiler Hints & Cache Policy — +4.9%

Disable LLVM loop-strength-reduction (`LSR=off`) to avoid register spills on high-occupancy kernels. Asymmetric L2 cache policy: weights `evict_last` (reused across M-tiles), tokens `evict_first` (streamed once).

6. Deep Pipeline + 2D Reduce — +5%

GEMM2 `num_stages=6--8` hides HBM latency (+2.1%). 2D tiled reduce (BLOCK_M×BLOCK_N) cuts grid >10× vs. 1D row-per-CTA (+2.9% large-T).

Precision Hierarchy



Intermediate (GEMM1→GEMM2): requires ≥FP16. SwiGLU spans ±40K (fp8 max = 448); bf16 passes 8/19 only. Scaling ×0.125/×8.0 clamps to FP16-safe range, halving HBM traffic.

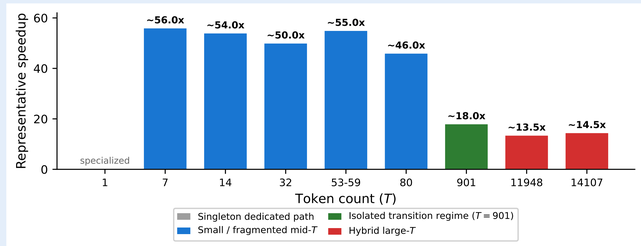
Expert output (GEMM2→reduce): bf16 OK (range 3.4×10^{38}); fp16 overflows 3/19. bf16 saves 50% write BW (+0.4%). Contest atol=1.0 accommodates bf16 precision loss.

MXFP8 (`tl.dot_scaled`): matched ratio 0.17–0.32. Shared exponents (UE8M0) too coarse for SwiGLU output distribution.

Headline Results

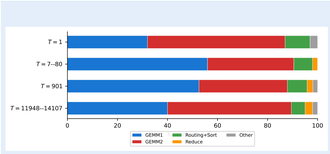
∼56× peak **∼54×** mean **19/19** correct

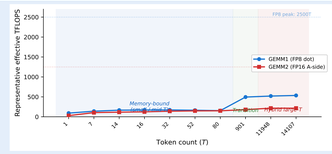
Per-Workload Speedup



Small/mid-T (7–80): ∼46–56× — baseline launch overhead dominates, fused pipeline amortizes it. Large-T (11,948–14,107): ∼13–15× — GEMM2 FP16 A-side prevents FP8 tensor core use. The gap reflects the GEMM1→GEMM2 bottleneck crossover: small-T is GEMM1-bound (FP8 dot helps), large-T is GEMM2-bound (FP16 A-side caps throughput).

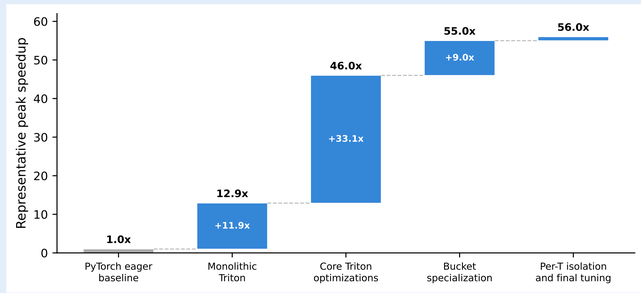
Kernel Time Breakdown & TFLOPS





Small T: GEMM1 dominates; expert weights repeatedly loaded for very few routed rows, with substantial padding waste.
Large T: GEMM2 becomes the main bottleneck; CuTe improves the grouped-GEMM core for the largest traces.
CPU overhead: Routing + sorting consumes ∼600 μs (56–73% wall time), but CUPTI scoring excludes it entirely.

Optimization Timeline (23 Rounds)



Largest jumps: eliminating per-expert launches, FP8-dot/non-atomic GEMM2 backbone, workload-aware specialization. Later gains are smaller but cumulative: compiler hints, deeper GEMM2 pipelining, bf16 expert_out, exact large-T dispatch, CuTe hybrid, and tight mid-T BLOCK_M tuning. Rounds 11–15 explored persistent kernels, TMA, warp specialization, fused reduce — all regressed or crashed.

Negative Results (8 Failed Attempts)

Approach	Result	Root Cause
FP8 Inter.	0/19	3-bit mantissa; SwiGLU > fp8 max
bf16 Inter.	8/19	Low mantissa + reg. pressure
Persistent	−1%	K=2048 too short for reuse
TMA Desc.	−4%	Setup > 16 K-loop iters
Warp Spec.	Crash	TTGIR lowering failure
Fused Red.	−4.5%	fp32 atomic 6× bf16 BW
CuTe mid-T	No gain	Metadata/sync not amortized
GEMM2 FP8	5/19	fp16→fp8 cast too lossy

Two failure groups: lower-precision buffers break correctness; persistence, TMA, fusion, and mid-T CuTe do not amortize setup for K=2048 (only 16 K-loop iterations).

Key Takeaways

1. **Understand scoring:** CUPTI GPU-only timing hides ∼600 μs CPU overhead—don't optimize what isn't measured
2. **Precision ≠ spectrum:** FP16 passes 19/19 (+4.5%); bf16 Intermediate fails 11/19; fp8 fails all—mantissa bits matter more than range
3. **A/B test everything:** ±15% per-workload noise demands same-session comparison (>2% threshold)
4. **Profile first:** All 8 successful techniques identified via NCU profiling, not speculation
5. **Tolerance matters:** Contest atol=1.0 unlocks bf16 expert output (+0.4% mean) that fails strict atol=0.01

Future Work

- Small-T memory-boundedness: 5.6% of FP8 peak at T=7 (expert weight cold misses)
- L2 persistent cache and epsilon expert skipping: prototyped but disabled pending tuning
- MXFP8 `tl.dot_scaled`: matched_ratio = 0.17–0.32 (shared exponents too coarse for GEMM2)
- Workload-adaptive kernel switching for production MoE serving

Measurement Noise

Modal B200 shared instances exhibit ±2% mean noise across 19 workloads, with ±15% per-workload variance for small T (<0.2 ms kernels) and <1% for large T (>2 ms). Cross-session drift reaches ±20−30% from co-tenant GPU interference. Our methodology: same-session A/B comparison with 2% mean threshold — only optimizations exceeding this noise floor are retained. Over 23 rounds, this discipline rejected 8 approaches that initially appeared promising (see Negative Results). Contest tolerance (atol=1.0) enables bf16 expert output (abs errors up to ∼600K) that would fail strict validation (atol=0.01).

References

[1] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437, 2024.
[2] Tillet et al., "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," MLSys 2019.
[3] Darvish Rouhani et al., "Microscaling Data Formats for Deep Learning," arXiv:2310.10537, 2023.
[4] Ye et al., "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," arXiv:2501.01005, 2025.
[5] Kerr et al., "CUTLASS: Fast Linear Algebra in CUDA C++," NVIDIA, 2017–2025.